

Call Stack e Event Loop

Usando Promise no NodeJS

- Retorno padrão de funções **async**
- Valores definidos com **resolve**
- Erros definidos com **reject**
- Captura do valor de resolve com **await** ou **then**
- Captura de erros com **catch**

```
const dividir = (a,b) => {
  const promise = new Promise((resolve,
reject)=>{
    if(b==0){ reject("Divisao por zero"); }
    else { resolve(a/b); }
  });
  return promise;
}
for(let i = 2; i>=-2; i--)
  dividir(10,i).then((x)=>console.log(x))

.catch((error)=>console.log(error));
```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  AZURE  powershell  + v  [ ] [ ] ... ^ X

PS D:\NodeProjs\Exemplo05> node index.js
Divisao por zero
10
20
-20
-10
PS D:\NodeProjs\Exemplo05>

```

```

const dobrar = (x) =>
  Promise.resolve(x*2);
for(let i = 2; i>=-2; i--)
  dividir(10,i).then((x1)=>dobrar(x1))

  .then((x2)=>console.log(x2))

  .catch((error)=>console.log(error));

```